

Finals Task 2. Inheritance

Problem School Performance

Note: You are to create 4 separate python files for this task:

- performer.py(base class)
- singer.py(sub class)
- dancer.py(sub class)
- test_class.py – following the required test cases

In a school musical performance, different types of performers participate. For this program, we will be implementing the performers.

Base Class - Performer:

- Properties:
 - `name` (type: str): Represents the name of the performer.
 - `age` (type: int): Represents the age of the performer.
- Constructor:
 - `__init__(self, name: str, age: int)`: Initializes the `name` and `age` properties.
- Getters
 - `get_name(self) -> str`: Returns the name
 - `get_age(self) -> int`: Returns the age

Subclass - Singer:

- Inherits From: `Performer`
- Additional Property:
 - `vocal_range` (type: str): Represents the vocal range of the singer.
- Constructor:
 - `__init__(self, name: str, age: int, vocal_range: str)`: Initializes the `name` and `age` properties by calling the parent class's constructor and sets the `vocal_range` property.
- Getter:
 - `get_vocal_range(self) -> str`: Returns the vocal range of the singer.
- Method:
 - `sing(self) -> None`: Prints "{name} is singing with a {vocal_range} range."

Source code:

```

3
4 class Performer:
5     def __init__(self, name: str, age: int):
6         self.name = name
7         self.age = age
8
9     def get_name(self) -> str:
10         return self.name
11
12     def get_age(self) -> int:
13         return self.age
14
15
16
17 class Singer(Performer):
18     def __init__(self, name: str, age: int, vocal_range: str):
19         super().__init__(name, age)
20         self.vocal_range = vocal_range
21
22     def get_vocal_range(self) -> str:
23         return self.vocal_range
24
25     def sing(self) -> None:
26         print(f"{self.name} is singing with a {self.vocal_range} range.")
27
28
29
30 class Dancer(Performer):
31     def __init__(self, name: str, age: int, dance_style: str):
32         super().__init__(name, age)
33         self.dance_style = dance_style
34
35     def get_dance_style(self) -> str:
36         return self.dance_style
37
38     def dance(self) -> None:
39         print(f"{self.name} is performing {self.dance_style} dance.")
40
41
42
43
44 print("Test Case 1")
45 p = Performer("John", 25)
46 print([p.get_name(), p.get_age()])
47 print()
48
49 print("Test Case 2")
50 d = Dancer("Emily", 28, "Ballet")
51 print([d.get_name(), d.get_age(), d.get_dance_style()])
52 print()
53
54 print("Test Case 3")
55 d.dance()
56 print()
57
58 print("Test Case 4")
59 print(issubclass(Dancer, Performer))
60 print()
61
62 print("Test Case 5")
63 s = Singer("Linda", 35, "Soprano")
64 print([s.get_name(), s.get_age(), s.get_vocal_range()])
65 print()
66
67 print("Test Case 6")
68 s.sing()
69 print()
70

```

Sample output:

```
Output
Test Case 1
['John', 25]

Test Case 2
['Emily', 28, 'Ballet']

Test Case 3
Emily is performing Ballet dance.

Test Case 4
True

Test Case 5
['Linda', 35, 'Soprano']

Test Case 6
Linda is singing with a Soprano range.
```